Laboratory 3:

Synthesis and Static Timing Analysis (STA)

OBJECTIVES

- The objective of this lab is for students to perform the complete digital design flow from RTL design → synthesis → gate-level simulation using **Genus** and **Xcelium** to carry out the synthesis and verification of a digital circuit.
- Using **Genus**, the RTL design written in HDL is synthesized into a **gate-level netlist** based on a standard cell library, while also generating corresponding timing information in the form of a **Standard Delay Format (SDF)** file and other parameters such as **power, area, and cell usage**.
- Know how to write an **SDC (Synopsys Design Constraints)** file using Tcl commands to **define constraint parameters** for digital designs. Perform timing report analysis after synthesis to verify the system's timing conditions.

PREPARATION FOR LAB 3

- Understand the concept and flow of the synthesis process in digital circuit design.
- Read the server access guide and the user guides for the Xcelium, SimVision and Genus tools.
- Understand how to write and apply TCL (Tool Command Language) scripts to automate the synthesis process in Genus.
- Know how to write an SDC file, which is a plain text file using Tcl syntax to define timing, power, and area constraints for digital designs.

REFERENCE

Laboratory 3:

Synthesis and Static Timing Analysis (STA)

EXPERIMENT 1

Objective: Perform timing analysis for combinational circuits after synthesis.

Description: In this experiment, we perform static timing analysis (STA) for a combinational design. The code used in this experiment is a code segment describing an ALU:

```
module alu (
                    input  logic [3:0] A,
                    input  logic [3:0] B,
                    input  logic [1:0]   S,
                    output logic [3:0] Y);

   always_comb begin
      case (S)
         2'b00: S=A+B;
         2'b01: S=A-B;
         2'b10: S=A&B;
         2'b11: S=A^B;
         default: S=4'd0;
      endcase
   end

endmodule
```

In this experiment, we will use standard cells under four different process corners in 45nm technology. The conditions for these eight corners are provided in Table 1.

Library Name	Process Corner	VDD (V)	Threshold Type
fast_vdd1v0_basicCells_hvt.lib	Fast (FF)	1.0 V	HVT (High-Vt)
fast_vdd1v2_basicCells_hvt.lib	Fast (FF)	1.2 V	HVT (High-Vt)
slow_vdd1v0_basicCells_hvt.lib	Slow (SS)	1.0 V	HVT (High-Vt)
slow_vdd1v2_basicCells_hvt.lib	Slow (SS)	1.2 V	HVT (High-Vt)

Table 1: Operating conditions of the four corners analyzed.

Instruction:

Students perform the synthesis steps as in Lab 2, with the following additional notes.

1. Add the file constraints.sdc with the following content:

```
set_max_delay 0.1 -from [all_inputs] -to [all_outputs]
set_input_delay 0.0 -clock [get_clocks *] [all_inputs]
set_output_delay 0.0 -clock [get_clocks *] [all_outputs]
set timing_report_unconstrained true
```

Meaning:

- set_max_delay 0.1 -from [all_inputs] -to [all_outputs]: Maximum path delay constraint between every input and every output of the circuit. The tool will attempt to keep the delay ≤ 0.1 ns (or 0.1 time unit of the design — typically ns).
- set_input_delay 0.0 -clock [get_clocks *] [all_inputs]: All input signals arrive at the circuit immediately at the clock edge (no external delay).
- set_output_delay 0.0 -clock [get_clocks *] [all_inputs]: The circuit must produce its outputs right at the clock edge, without assuming any external slack.
- set timing_report_unconstrained true: When you run report_timing, the tool will also report any unconstrained paths.

2. Write a TCL script for Cadence Genus to synthesize the alu circuit from the alu.sv file. The script is shown. In this step, remember to read constraints file (sdc file). Use the command *report_timing* to print the worst path, and the command *report_timing -nworst 10* to print the 10 worst paths.

```
set_db  init_lib_search_path {… }              #Path to library
set_db  init_hdl_search_path {… }              #Path to HDL source code folder
read_libs  slow_vdd1v2_basicCells_lvt.lib   #Read library corner

read_hdl  -sv  alu.sv                          #Read HDL file
elaborate

read_sdc  constraints.sdc                      #Read constraints file

set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium

syn_generic
syn_map
syn_opt                  ---> We use this command in this experiment
```

```
#Generate reports
report_timing  > 02_reports/timing.rpt
report_timing -nworst 10  >02_reports/timing_10_path.rpt
report_power   > 02_reports/power.rpt
report_area    > 02_reports/area.rpt
report_qor     > 02_reports/qor.rpt

#Generate netlist and sdf file
write_hdl      > 03_outputs/and2_netlist.sv
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge -setuphold split  > 03_outputs/delay.sdf
```

3. Students can read the report files timing.rpt and timing_10_path.rpt to view the worst path and the 10 worst paths.

Requirements:

- Find the worst path using the conditions specified in the provided SDC file: indicate its traversal and draw the worst path from the synthesis results.
- Perform timing analysis (you may draw timing diagrams) of the 10 worst paths. For each path, indicate its traversal (no drawing needed), the path delay, and specify whether the path meets slack or is violated.

Perform the above requirements for each corner in the 4 corners mentioned above.

Laboratory 3:

Synthesis and Static Timing Analysis (STA)

EXPERIMENT 2

Objective: Perform timing analysis for combinational circuits after synthesis.

Description: In this experiment, we investigate how the Genus tool optimizes a combinational circuit after synthesis according to the timing constraints specified in the SDC file.

The contents of the constraints.sdc file used in this experiment are shown below:

```
set_max_delay 10 -from [all_inputs] -to [all_outputs]
set_input_delay 0.0 -clock [get_clocks *] [all_inputs]
set_output_delay 0.0 -clock [get_clocks *] [all_outputs]
set timing_report_unconstrained true
```

Requirements:

Use the library *slow_vdd1v2_basicCells_hvt.lib* to do the following requirements:

- Perform synthesis of the code in Experiment 1 using a maximum path delay constraint of 10 ns.
- Reduce the maximum timing constraint to 1 ns, 0.5 ns, 0.2 ns, and 0.1 ns. At each timing point, perform synthesis of the code and record the results.

Your report has to show the results:

- Indicate the differences in the synthesized circuit when changing the timing constraint conditions. Summarize the synthesis behavior when decreasing the maximum timing constraint.
- At each timing constraint point: capture the synthesized circuit result and describe the longest path (draw it on the circuit and state whether the path meets the slack or is violated).

Laboratory 3:

Synthesis and Static Timing Analysis (STA)

EXPERIMENT 3

Objective: Perform timing analysis for sequential circuits after synthesis.

Description: In this experiment, we perform static timing analysis (STA) for a sequential design. The code used in this experiment is a code segment describing an ALU:

```
module alu (
                    input  logic [3:0] A,
                    input  logic [3:0] B,
                    input  logic [1:0]   S,
                    input logic clk,
                    output logic [3:0] Y);

   always_ff @ (posedge clk) begin
      case (S)
         2'b00: S=A+B;
         2'b01: S=A-B;
         2'b10: S=A&B;
         2'b11: S=A^B;
         default: S=4'd0;
      endcase
   end

endmodule
```

Instruction:

Students perform the synthesis steps as in Lab 2 or in Experiment 1 of this Lab, with the following additional notes.

Add the file constraints.sdc with the following content:

```
create_clock -name clk -period 1.0 [get_ports clk]


set_input_delay 0.2 -clock [get_clocks *] [all_inputs]
set_output_delay 0.3 -clock [get_clocks *] [all_outputs]


set_max_delay 0.5 -from [all_inputs] -to [all_outputs]


set timing_report_unconstrained true
```

Meaning:

- create_clock -name clk -period 1.0 [get_ports clk]: The tool will create a clock object with the logical name clk and define the clock period as 1.0 ns. The clk signal from the top-level input port is considered the clock source.
- set_max_delay 0.5 -from [all_inputs] -to [all_outputs]: Maximum path delay constraint between every input and every output of the circuit. The tool will attempt to keep the delay ≤ 0.5 ns (or 0.5 time unit of the design — typically ns).
- set_input_delay 0.2 -clock [get_clocks *] [all_inputs]: All input signals arrive at the circuit 0.2 ns after the clock edge.
- set_output_delay 0.3 -clock [get_clocks *] [all_inputs]: The data at the output port must be stable at the off-chip side 0.3 ns before the clock edge..
- set timing_report_unconstrained true: When you run report_timing, the tool will also report any unconstrained paths.

Requirements:

Use the library *slow_vdd1v2_basicCells_hvt.lib* to do the following requirements:

- Find the worst path using the conditions specified in the provided SDC file: indicate its traversal and draw the worst path from the synthesis results.
- Perform timing analysis (you may draw timing diagrams) of the 10 worst paths. For each path, indicate its traversal (no drawing needed), the path delay, and specify whether the path meets slack or is violated.
- Modify the SDC constraint parameters according to the following table:

Scenario	period of clk	max_delay	input_delay	output_delay
1	1ns	0.2ns	0.2ns	0.3ns
2	1ns	0.2ns	0.1ns	0.3ns
3	0.8ns	0.2ns	0.3ns	0.3ns
4	0.6ns	0.2ns	0.2ns	0.2ns
5	0.5ns	0.2ns	0.1ns	0.2ns

Table 2: Table of constraint parameters used during synthesis.

At each timing constraint point: capture the synthesized circuit result and describe the longest path (draw it on the circuit and state whether the path meets the slack or is violated).

Laboratory 3:

Synthesis and Static Timing Analysis (STA)

EXPERIMENT 4

Objective: Perform timing analysis for the low-power sequential circuit after synthesis.

Description: In this experiment, we perform STA for a low-power sequential design. The code used in this experiment is a code segment describing an ALU that employs clock-gating techniques:

```
module alu_low_power (
    input  logic [3:0] A,
    input  logic [3:0] B,
    input  logic [1:0] S,
    input  logic       clk,
    output logic [3:0] Y
);

    logic [3:0] add_res, sub_res, and_res, xor_res;
    logic en_add,en_sub,en_and,en_xor;

    // Activate only the necessary logic to reduce switching
    // Generate enable signals for each block
    assign en_add = (S == 2'b00)?1'b1:1'b0;
    assign en_sub = (S == 2'b01)?1'b1:1'b0;
    assign en_and = (S == 2'b10)?1'b1:1'b0;
    assign en_xor = (S == 2'b11)?1'b1:1'b0;

    // Use clock gating to supply clock signals to each logic block
    logic clk_add, clk_sub, clk_andl, clk_xorl;
    // Clock gating using an AND gate
    assign clk_add  = clk & en_add;
    assign clk_sub  = clk & en_sub;
    assign clk_andl = clk & en_and;
    assign clk_xorl = clk & en_xor;

    // These blocks operate only when en = 1.
```

```
    always_ff @(posedge clk_add)
        add_res <= A + B;

    always_ff @(posedge clk_sub)
        sub_res <= A - B;

    always_ff @(posedge clk_andl)
        and_res <= A & B;

    always_ff @(posedge clk_xorl)
        xor_res <= A ^ B;

    //Output Selection Mux
    always_comb begin
        case (S)
            2'b00: Y = add_res;
            2'b01: Y = sub_res;
            2'b10: Y = and_res;
            2'b11: Y = xor_res;
            default: Y = 4'd0;
        endcase
    end

endmodule
```

Requirements:

Use the library *slow_vdd1v2_basicCells_hvt.lib* to do the following requirements:

- Synthesize the code in the description with constraints in Experiment 3.

```
create_clock -name clk -period 1.0 [get_ports clk]

set_input_delay 0.2 -clock [get_clocks *] [all_inputs]
set_output_delay 0.3 -clock [get_clocks *] [all_outputs]
```

```
set_max_delay 0.5 -from [all_inputs] -to [all_outputs]

set timing_report_unconstrained true
```

Compare the power consumption of the two designs from Experiments 3 and 4. Explain why the power consumption differs and why it is lower.

- Find the worst path using the conditions specified in the provided SDC file: indicate its traversal and draw the worst path from the synthesis results.
- Perform timing analysis (you may draw timing diagrams) of the 10 worst paths. For each path, indicate its traversal (no drawing needed), the path delay, and specify whether the path meets slack or is violated.